

P3055R0

Relax wording to permit relocation optimizations in the STL

Published Proposal, 2023-12-09

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:

3

Abstract

We catalog the STL containers and algorithms that could benefit from optimizations related to relocation (P1144, P2786), and propose to loosen their current overspecification so that such optimizations become legal (but not mandatory), even in the absence of a standardized vocabulary for relocation.

Table of Contents

1 Changelog

2 Motivation and proposal

2.1 Benchmark

3 Breakage of existing code

4 Implementation experience

5 Proposed wording

5.1 [vector.modifiers]

5.2 [deque.modifiers]

5.3 [alg.rotate]

5.4 [alg.swap]

5.5 [utility.swap] (unchanged)

5.6 [iterator.cust.swap] (unchanged)

References

Informative References

§ 1. Changelog

- R0:
 - Initial revision.

§ 2. Motivation and proposal

We have two largely similar active proposals for "trivial relocation" in C++: Arthur's [\[P1144\]](#), and Bloomberg's [\[P2786\]](#) / [\[P2959\]](#) / [\[P2967\]](#). We also have two recent proposals for "non-trivial relocation" as a new fundamental operation: Bloomberg's [\[P2839\]](#) (rejected by EWG in Varna) and Bini and Catmur's [\[P2785\]](#) (still active).

A major motivation for "(trivial) relocation" is that it allows library authors to choose "fast paths" for (trivially) relocatable types. For example, `vector::erase(it)` for non-relocatable `T` must use `operator=` in a loop (represented here by `std::move`), but for trivially relocatable `T` it can avoid `operator=` and use the moral equivalent of `memcpy` (represented here by P1144 `std::uninitialized_relocate`).

```
void erase(iterator it) {
    if constexpr (std::is_trivially_relocatable_v<value_type>) {
        std::destroy_at(it);
        std::uninitialized_relocate(it + 1, end_, it);
    } else {
        std::move(it + 1, end_, it); // operator=
        std::destroy_at(end_ - 1);
    }
    --end_;
}
```

The exact details of that snippet are up for debate: should the library provide a trait `is_nothrow_relocatable_v`? should `uninitialized_relocate` be guaranteed, or merely encouraged, to use `memcpy` instead of move-and-destroy-in-a-loop? and so on. This paper P3055 considers all those little details to be "out of scope"; they don't affect the gist of this paper.

This paper concerns itself with one giant problem — anything like the above implementation is, formally, forbidden by the current Standard! To permit the above implementation, we must loosen the specification of `vector::erase` ([[vector.modifiers](#)]/3) along these lines:

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();
```

3. *Effects*: Invalidates iterators and references at or after the point of the erase.
4. *Throws*: Nothing unless an exception is thrown by the assignment operator or move assignment operator of `T`.
5. *Complexity*: ~~The destructor of `T` is called the number of times equal to the number of the elements erased, but the assignment operator of `T` is called the number of times equal to the number of elements in the vector after the erased elements.~~ Linear in the number of elements following the first erased element in the original vector.

This change would be consistent with LWG's wording choices in the modern era: it specifies the complexity only in terms of big-O, and does not directly mandate any particular implementation strategy. So for example it would become legal for the implementation to do just this:

```
void erase(iterator it) {
    std::rotate(it, it + 1, end_);
    std::destroy_at(end_--);
}
```

according to `std::rotate`'s current wording. But furthermore, we propose to loosen `std::rotate`'s wording ([\[alg.rotate\]](#)) too:

1. *Preconditions*: `[first, middle)` and `[middle, last)` are valid ranges. For the overloads in namespace `std`, `ForwardIterator` meets the *Cpp17ValueSwappable* requirements ([\[swappable.requirements\]](#)), and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
2. *Effects*: For each non-negative integer $i < (last - first)$, places the element from the position `first + i` into position `first + (i + (last - middle)) % (last - first)`. *[Note: This is a left rotate. — end note]*
3. *Returns*:
 - `first + (last - middle)` for the overloads in namespace `std`.
 - `{first + (last - middle), last}` for the overload in namespace `ranges`.
4. *Complexity*: ~~At most $last - first$ swaps.~~ Linear in $last - first$.

`std::rotate`'s previous wording was defined in terms of "swaps." Look at the specification for `std::swap` ([\[utility.swap\]](#)):

```
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below);
```

1. *Constraints*: `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.
2. *Preconditions*: Type `T` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
3. *Effects*: Exchanges values stored in two locations.
4. *Remarks*: The exception specification is equivalent to: `is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>`.

Here the status quo is sufficiently loose to permit an efficient implementation by means of relocation, and in fact Arthur's libc++ fork does exactly that ([source](#), [Godbolt](#)). The following code omits details such as `std::is_constant_evaluated()` which are present in the actual library.

```
void swap(T& a, T& b)
    noexcept(is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>)
{
    if constexpr (std::is_trivially_relocatable_v<T>) {
        __builtin_memswap(&a, &b, __datasizeof(T));
    } else {
        T temp = std::move(a);
        a = std::move(b);
        b = std::move(temp);
    }
}
```

In other words, this paper P3055 is needed, even after P1144/P2786. If one of those papers is adopted without P3055, then conforming implementations will still be technically forbidden to do the optimizations we want to enable (which are already done by `bsl::vector`, `folly::FBVector`, etc). Vice versa, as soon as P3055 is adopted (even without P1144/P2786), a conforming implementation will be able to use these optimizations on types it knows to be trivially relocatable (e.g. `std::deque<int>`), even if we continue to lack a standardized vocabulary for relocatable user-defined types.

§ 2.1. Benchmark

Trivial relocation is an "infinitely valuable optimization" in the same sense as C++11 move semantics. For the following type `S`, mainline libc++ compiles `std::swap` into 74 lines of assembly. Arthur's P1144 fork of libc++ compiles it into 18 lines. ([Godbolt](#).)

```
struct S {
    S();
    std::unique_ptr<int> p;
    std::shared_ptr<int> q;
    bool b;
};
```

```
void test(S& a, S& b) {
    std::swap(a, b);
}
```

This propagates back up the call-stack as high as we're willing to let it propagate. Arthur's libc++ effectively applies this paper P3055's proposed wording already, permitting `rotate` to be implemented in terms of `swap` and `erase` to be implemented in terms of `rotate`.

Operation	Mainline libc++ LOC	P1144 libc++ LOC
<code>std::swap(S&, S&)</code>	74	18
<code>std::rotate(S*, S*, S*)</code>	145	122
<code>vector<S>::erase(it)</code>	108	39

§ 3. Breakage of existing code

Since the proposed wording is looser than the existing wording, all vendors already conform to it by definition; we're not asking any vendor to change their implementation for C++26. However, a vendor who takes advantage of the new freedom may change the behavior of certain algorithms and containers for non-regular types. Following [P2959], we'll use `tuple<int&>` as our canonical example. `tuple<int&>` assigns-through on assignment and swap; it never rebinds (except on initial construction). This is the polar opposite of `reference_wrapper<int>`, which rebinds on assignment and swap, and never assigns-through. In P1144's terminology, `reference_wrapper<int>` is trivially relocatable and `tuple<int&>` is not trivially relocatable. (Godbolt.)

Recall that `swap` is already loosely specified — it "exchanges the values" of its arguments — so our proposal leaves the following example untouched:

```
int i = 1, j = 2;
std::tuple<int&> a = {i}, b = {j};
std::swap(a, b);
assert(i == 2 && j == 1);
```

`std::rotate`'s *Effects* are specified via the phrase "places the element from position x into position y "; its semantics are coupled to `swap` only through the phrase "At most n swaps" in the *Complexity* element, which we propose to remove. After that change, a vendor might reasonably construe that this old behavior...

```
int a[3] = {1,2,3};
std::tuple<int&> ts[3] = {{a[0]}, {a[1]}, {a[2]}};
std::rotate(ts, ts+1, ts+3);
assert(a[0] == 2 && a[1] == 3 && a[2] == 1);
```

...was no longer strictly mandated. They might choose to "place" the `tuple<int&>`s as-if-by relocation, rebinding each `tuple<int&>` and leaving the array `a` untouched. (However, Arthur's libcpp doesn't change this behavior, because Arthur's libcpp optimizes only trivially relocatable types, and `tuple<int&>` is not trivially relocatable.)

Consider `vector::erase`, whose semantics are coupled to `operator=` only through wording in its *Complexity* element which we propose to remove. After that change, a vendor might reasonably construe that this old behavior...

```
int a[3] = {1,2,3};
std::vector<std::tuple<int&>> ts = {{a[0]}, {a[1]}, {a[2]}};
ts.erase(ts.begin());
assert(a[0] == 2 && a[1] == 3 && a[2] == 3);
```

...was no longer strictly mandated. They might choose to "erase the element pointed to" ([[sequence.reqmts/47](#)]) as-if-by relocation, rebinding each `tuple<int&>` and leaving the array `a` untouched. As [[P2959](#)] points out, this is exactly what happens anyway if you switch out `vector` for `list`. (Again, Arthur's libcpp doesn't change this behavior, because `tuple<int&>` is not trivially relocatable; but we certainly have no desire to continue mandating the old behavior.)

§ 4. Implementation experience

Since the proposed wording is looser than the existing wording, all vendors already conform to it by definition; we're not asking any vendor to change their implementation for C++26.

Arthur has implemented trivial-relocation optimizations in his fork of libcpp, and used it to compile both LLVM/Clang/libcpp and another large C++17 codebase. No problems were found (naturally).

§ 5. Proposed wording

NOTE: We're trying to eliminate places where the *Effects* and *Complexity* elements specifically mention assignment. We don't mind e.g. when [deque.modifiers] specifies that `push_back` "causes a single call to a constructor of `T`," because that's still correct even if we're optimizing trivially relocatable types. We don't even mind when [vector.modifiers] specifies that `erase` calls `T`'s destructor "the number of times equal to the number of the elements erased," because of course it does; but we propose to remove that sentence anyway because it is redundant. We also don't mind when an operation says "*Throws*: Nothing unless an exception is thrown from the assignment operator of `T`," because our new trivial-relocation "happy path" will never throw. Such a *Throws* element continues to describe the circumstances under which the operation *might* throw. We never propose to loosen any *Throws* element.

§ 5.1. [vector.modifiers]

Modify [vector.modifiers] as follows:


```

constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);

template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... a
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);

```

1. *Complexity*: If reallocation happens, linear in the number of elements of the resulting vector; otherwise, linear in the number of elements inserted plus the distance to the end of the vector.

2. *Remarks*: Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. If no reallocation happens, then references, pointers, and iterators before the insertion point remain valid but those at or after the insertion point, including the past-the-end iterator, are invalidated. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T or by any `InputIterator` operation there are no effects. If an exception is thrown while inserting a single element at the end and T is `Cpp17CopyInsertable` or `is_nothrow_move_constructible_v<T>` is true, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`Cpp17CopyInsertable` T, the effects are unspecified.

```

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();

```

3. *Effects*: Invalidates iterators and references at or after the point of the erase.

4. *Throws*: Nothing unless an exception is thrown by the assignment operator or move assignment operator of T.

5. *Complexity*: ~~The destructor of T is called the number of times equal to the number of the elements erased, but the assignment operator of T is called the number of times equal to the number of elements in the vector after the erased elements.~~ Linear in the number of elements after the first erased element in the original vector.

§ 5.2. [deque.modifiers]

Modify [deque.modifiers] as follows:

```

iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    iterator insert(const_iterator position,
                    InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    iterator insert_range(const_iterator position, R&& rg);
iterator insert(const_iterator position, initializer_list<T>);

template<class... Args> reference emplace_front(Args&&... args);
template<class... Args> reference emplace_back(Args&&... args);
template<class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
template<container-compatible-range<T> R>
    void prepend_range(R&& rg);
void push_back(const T& x);
void push_back(T&& x);
template<container-compatible-range<T> R>
    void append_range(R&& rg);

```

1. *Effects*: An insertion in the middle of the deque invalidates all the iterators and references to elements of the deque. An insertion at either end of the deque invalidates all the iterators to the deque, but has no effect on the validity of references to elements of the deque.

2. *Complexity*: The complexity is linear Linear in the number of elements inserted plus the lesser of the distances to the beginning and end of the deque. Inserting a single element at either the beginning or end of a deque always takes constant time and causes a single call to a constructor of T.

3. *Remarks*: If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T there are no effects. If an exception is thrown while inserting a single element at either end, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-*Cpp17CopyInsertable* T, the effects are unspecified.

```

iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_front();
void pop_back();

```

4. *Effects*: An erase operation that erases the last element of a deque invalidates only the past-the-end iterator and all iterators and references to the erased elements. An erase operation that erases the first element of a deque but not the last element invalidates only iterators and references to the erased elements. An erase operation that erases neither the first element nor the last element of a deque invalidates the past-the-end iterator and all iterators and references to all the elements of the deque. [Note: pop_front and pop_back are erase operations. — end note]

5. *Throws*: Nothing unless an exception is thrown by the assignment operator of T.

6. *Complexity*: ~~The number of calls to the destructor of T is the same as the number of elements erased, but the number of calls to the assignment operator of T is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.~~ Linear in the lesser of the number of elements after the first erased element and the number of elements before the last erased element in the original deque.

§ 5.3. [alg.rotate]

Modify [alg.rotate] as follows:

```
template<class ForwardIterator>
constexpr ForwardIterator
    rotate(ForwardIterator first, ForwardIterator middle, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    rotate(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator middle, ForwardIterator last);
template<permutable I, sentinel_for<I> S>
constexpr subrange<I> ranges::rotate(I first, I middle, S last);
```

1. *Preconditions*: $[first, middle)$ and $[middle, last)$ are valid ranges. For the overloads in namespace `std`, `ForwardIterator` meets the `Cpp17ValueSwappable` requirements ([swappable.requirements]), and the type of $*first$ meets the `Cpp17MoveConstructible` (Table 31) and `Cpp17MoveAssignable` (Table 33) requirements.

2. *Effects*: For each non-negative integer $i < (last - first)$, places the element from the position $first + i$ into position $first + (i + (last - middle)) \% (last - first)$. [Note: This is a left rotate. — end note]

3. Returns:

- $first + (last - middle)$ for the overloads in namespace `std`.
- $\{first + (last - middle), last\}$ for the overload in namespace `ranges`.

4. *Complexity*: ~~At most $last - first$ swaps.~~ Linear in $last - first$.

§ 5.4. [alg.swap]

Modify [alg.swap] as follows:

```

template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator2
    swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
                ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    swap_ranges(ExecutionPolicy&& exec,
                ForwardIterator1 first1, ForwardIterator1 last1,
                ForwardIterator2 first2);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2>
requires indirectly_swappable<I1, I2>
constexpr ranges::swap_ranges_result<I1, I2>
    ranges::swap_ranges(I1 first1, S1 last1, I2 first2, S2 last2);
template<input_range R1, input_range R2>
requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
constexpr ranges::swap_ranges_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
    ranges::swap_ranges(R1&& r1, R2&& r2);

```

1. Let:

- $last2$ be $first2 + (last1 - first1)$ for the overloads with no parameter named $last2$;
- M be $\min(last1 - first1, last2 - first2)$.

2. *Preconditions*: The two ranges $[first1, last1)$ and $[first2, last2)$ do not overlap. For the overloads in namespace `std`, $*(first1 + n)$ is swappable with $([swappable.requirements])*(first2 + n)$ for all n in the range $[0, M)$.

3. *Effects*: For each non-negative integer $n < M$ performs:

- ~~`swap(*(first1 + n), *(first2 + n))`~~ exchanges the values of $*(first1 + n)$ and $*(first2 + n)$ for the overloads in namespace `std`;
- performs `ranges::iter_swap(first1 + n, first2 + n)` for the overloads in namespace `ranges`.

4. *Returns*:

- $last2$ for the overloads in namespace `std`.
- $\{first1 + M, first2 + M\}$ for the overloads in namespace `ranges`.

5. *Complexity*: Exactly M swaps.

```

template<class ForwardIterator1, class ForwardIterator2>
constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

```

6. *Preconditions*: a and b are dereferenceable. $*a$ is swappable with $([swappable.requirements])*b$.

7. *Effects*: As if by `swap(*a, *b)`.

§ 5.5. [utility.swap] (unchanged)

[utility.swap] does not seem to require any changes:

```
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below);
```

1. *Constraints:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.
2. *Preconditions:* Type `T` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
3. *Effects:* Exchanges values stored in two locations.
4. *Remarks:* The exception specification is equivalent to: `is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>`

```
template<class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);
```

5. *Constraints:* `is_swappable_v<T>` is true.
6. *Preconditions:* `a[i]` is swappable with `([swappable.requirements]) b[i]` for all `i` in the range `[0, N)`.
7. *Effects:* As if by `swap_ranges(a, a + N, b)`.

§ 5.6. [iterator.cust.swap] (unchanged)

NOTE: Trivial types may ignore the first bullet below, because even if a user-defined ADL `iter_swap` is available, it must "exchange the values denoted by `E1` and `E2`" — that is, it must not be observably different from an ordinary (possibly trivial) swap. The second and third bullets explicitly describe ways of performing an ordinary (possibly trivial) swap by hand; for any P1144 trivially relocatable type, these are guaranteed to be tantamount to swapping the bytes. Therefore vendors can already optimize `ranges::iter_swap` today, without any change in this section's wording.

[iterator.cust.swap] does not seem to require any changes:

1. The name `ranges::iter_swap` denotes a customization point object ([`customization.point.object`]) that exchanges the values ([`concept.swappable`]) denoted by its arguments.

2. Let *iter-exchange-move* be the exposition-only function template:

```
template<class X, class Y>
constexpr iter_value_t<X> iter-exchange-move(X&& x, Y&& y)
    noexcept(noexcept(iter_value_t<X>(iter_move(x))) &&
             noexcept(*x = iter_move(y)));
```

3. *Effects*: Equivalent to:

```
iter_value_t<X> old_value(iter_move(x));
*x = iter_move(y);
return old_value;
```

4. The expression `ranges::iter_swap(E1, E2)` for subexpressions `E1` and `E2` is expression-equivalent to:

- `(void)iter_swap(E1, E2)`, if either `E1` or `E2` has class or enumeration type and `iter_swap(E1, E2)` is a well-formed expression with overload resolution performed in a context that includes the declaration

```
template<class I1, class I2>
void iter_swap(I1, I2) = delete;
```

and does not include a declaration of `ranges::iter_swap`. If the function selected by overload resolution does not exchange the values denoted by `E1` and `E2`, the program is ill-formed, no diagnostic required. [*Note*: This precludes calling unconstrained `std::iter_swap`. When the deleted overload is viable, program-defined overloads need to be more specialized ([`temp.func.order`]) to be selected. —*end note*]

- Otherwise, if the types of `E1` and `E2` each model `indirectly_readable`, and if the reference types of `E1` and `E2` model `swappable_with` ([`concept.swappable`]), then `ranges::swap(*E1, *E2)`.
- Otherwise, if the types `T1` and `T2` of `E1` and `E2` model `indirectly_movable_storable<T1, T2>` and `indirectly_movable_storable<T2, T1>`, then `(void)(*E1 = iter-exchange-move(E2, E1))`, except that `E1` is evaluated only once.
- Otherwise, `ranges::iter_swap(E1, E2)` is ill-formed. [*Note*: This case can result in substitution failure when `ranges::iter_swap(E1, E2)` appears in the immediate context of a template instantiation. —*end note*]

§ References

§ Informative References

[P1144]

Arthur O'Dwyer. *std::is_trivially_relocatable*. October 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p1144r9.html>

[P2785]

Sébastien Bini; Ed Catmur. *Relocating prvalues*. June 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2785r3.html>

[P2786]

Mungo Gill; Alisdair Meredith. *Trivial relocatability for C++26*. October 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2786r3.pdf>

[P2839]

Brian Bi; Joshua Berne. *Non-trivial relocation via a new owning reference type*. May 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2839r0.html>

[P2959]

Alisdair Meredith. *Relocation within containers*. October 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2959r0.html>

[P2967]

Alisdair Meredith. *Relocation has a library interface*. October 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2967r0.pdf>